INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: BASES DE DATOS

PRÁCTICA 1: MODELO ENTIDAD-RELACIÓN

Ejercicio 3
¿Qué es una base de datos?
Investigación de la Unidad Temática I

Profesor: Hurtado Avilés Gabriel

Integrantes

Damas Garnica Adrian

Esparza Ramón Fernando Adán

Ramos Padrón Jesus Emmanuel

Hernandez Barrios Samuel Rodrigo

Tercer semestre

Grupo: 3CV4

Fecha: 18 de septiembre de 2026

Ejercicio 3. ¿Qué es una base de datos? Investigación de la Unidad Temática I

1. Dato, información y base de datos

Es común usar los términos “dato” e “información” como sinónimos en el lenguaje cotidiano, pero en el campo de las bases de datos se distinguen con precisión. Date (2004) define un dato como un hecho registrado, un valor aislado sin contexto que lo interprete —por ejemplo, el número 45 no significa nada por sí mismo—, mientras que la información es el resultado de procesar esos datos de forma que adquieran significado para quien los recibe, de modo que 45 se convierte en información cuando se sabe que representa la edad de un paciente en un expediente clínico. Elmasri y Navathe (2016), por su parte, definen el dato de manera más operativa como un hecho conocido que puede registrarse y que tiene un significado implícito, y describen la base de datos como una colección de datos relacionados entre sí que representan algún aspecto del mundo real, recolectada con un propósito específico y diseñada, construida y poblada con datos para una finalidad determinada.

Comparando ambas posturas, Date pone el énfasis en la frontera conceptual entre dato e información —el dato como materia prima, la información como dato interpretado—, mientras que Elmasri y Navathe ponen el énfasis en la organización y el propósito: no basta con acumular datos relacionados, la base de datos existe porque fue diseñada intencionalmente para representar una porción acotada de la realidad (un “miniuniverso” o universo de discurso) y servir a un fin concreto. Ambas definiciones son compatibles y, tomadas en conjunto, permiten definir la base de datos, para efectos de este trabajo, como una colección estructurada y persistente de datos relacionados lógicamente entre sí, organizada con un propósito definido, de forma que su procesamiento permita obtener información útil para quien la consulta.

2. Características de una base de datos

Silberschatz et al. (2020) y Coronel y Morris (2019) coinciden en señalar un conjunto de propiedades que distinguen a un sistema de bases de datos de un simple conjunto de archivos. La integración se refiere a que los datos de distintas áreas de una organización se almacenan de forma unificada, evitando que cada departamento mantenga su propia copia aislada de la misma información. La persistencia implica que los datos permanecen almacenados de forma estable más allá de la ejecución del programa que los creó o consultó, a diferencia de una variable en memoria que desaparece al terminar el proceso.

La redundancia controlada acepta que cierta duplicación de datos pueda existir deliberadamente (por ejemplo, para optimizar consultas), pero gestionada y documentada, en contraste con la redundancia descontrolada de los sistemas de archivos tradicionales, donde la misma información se repite de forma inconsistente en varios lugares. La integridad garantiza que los datos cumplan las reglas de negocio y restricciones definidas (tipos de dato, llaves, valores permitidos), de modo que la base de datos no permita estados inválidos. La independencia de datos separa la forma en que los datos se almacenan físicamente de la forma en que las

aplicaciones los perciben lógicamente, permitiendo modificar el almacenamiento físico sin reescribir el software que lo consume.

La seguridad restringe el acceso a los datos únicamente a los usuarios autorizados, mediante mecanismos de autenticación y permisos diferenciados. La concurrencia permite que múltiples usuarios lean y escriban en la base de datos de forma simultánea sin que sus operaciones interfieran entre sí de manera incorrecta, típicamente mediante control de transacciones. Finalmente, la recuperación (recovery) asegura que, ante una falla de hardware, software o un corte de energía, la base de datos pueda restaurarse a un estado consistente, sin perder las transacciones que ya se habían confirmado.

3. Archivos frente a bases de datos

Antes de la consolidación de los sistemas gestores de bases de datos, las organizaciones almacenaban su información en archivos independientes, cada uno gestionado por un programa de aplicación específico —un archivo de nómina para el sistema de recursos humanos, otro de inventario para el sistema de almacén, sin relación formal entre ellos—. Este enfoque de archivos genera tres problemas estructurales que Elmasri y Navathe (2016) documentan con claridad. El primero es la redundancia: si el nombre de un empleado se necesita tanto en nómina como en el sistema de asistencia, ese dato se captura y almacena por duplicado en dos archivos distintos, gestionados por dos programas distintos.

El segundo problema es la inconsistencia, consecuencia directa del anterior: si el nombre del empleado se actualiza en un archivo pero no en el otro —por ejemplo, tras un cambio de estado civil que modifica el apellido—, los dos sistemas quedan desincronizados y ya no hay una única versión confiable del dato. El tercer problema es la dependencia programa-datos: en un sistema de archivos, la estructura física del archivo está firmemente ligada al código del programa que lo lee, de modo que cualquier cambio en la estructura de los datos —agregar un campo, cambiar un tipo— obliga a modificar y recompilar el programa de aplicación correspondiente. Estos tres problemas, acumulados a medida que crecían las organizaciones y el número de programas que manejaban datos relacionados, dieron origen a los sistemas gestores de bases de datos, cuyo propósito central es centralizar los datos, controlar su redundancia y desacoplar su estructura física de los programas que los utilizan mediante la independencia de datos.

4. Usuarios de una base de datos

Coronel y Morris (2019) distinguen varios roles de usuario, cada uno con responsabilidades y niveles de permiso distintos. El diseñador de bases de datos es responsable de identificar los datos que deben almacenarse y de construir el modelo conceptual (entidad-relación) y lógico que los estructura, típicamente antes de que exista una sola línea de código de la aplicación; su permiso relevante es sobre el esquema, no sobre los datos en producción. El administrador de bases de datos (DBA) es responsable de la instalación, configuración, seguridad, respaldo, afinación de desempeño y otorgamiento de permisos sobre el sistema gestor ya en operación; cuenta con el nivel de privilegio más alto, incluyendo la capacidad de crear y eliminar usuarios y bases de datos completas.

El programador de aplicaciones es responsable de escribir el código que interactúa con la base de datos mediante un lenguaje de manipulación de datos embebido o mediante un ORM, y su permiso suele limitarse a las operaciones necesarias para la aplicación que desarrolla (lectura y escritura sobre tablas específicas), sin privilegios administrativos. El usuario final, finalmente, interactúa con la base de datos exclusivamente a través de la interfaz de la aplicación —sin escribir consultas directamente— y su responsabilidad se limita al uso correcto del sistema; sus permisos están completamente mediados por la aplicación y no tiene acceso directo al gestor.

5. Ciclo de vida de una base de datos

El ciclo de vida de una base de datos, siguiendo a Elmasri y Navathe (2016), se compone de etapas secuenciales, cada una con un entregable concreto que alimenta a la siguiente. En la etapa de recolección de requerimientos y análisis se entrevista a los usuarios y áreas involucradas para identificar qué datos deben guardarse y qué operaciones deben soportarse; su entregable es un documento de especificación de requerimientos. En el diseño conceptual se traduce esa especificación en un modelo entidad-relación, independiente de cualquier sistema gestor particular; su entregable es el diagrama ER.

En el diseño lógico (o mapeo de modelo de datos) el modelo conceptual se transforma al modelo que soportará el gestor elegido —en el caso relacional, a un conjunto de tablas normalizadas—; su entregable es el esquema relacional. En el diseño físico se decide cómo se almacenarán físicamente esos datos: índices, particionamiento, tipos de almacenamiento; su entregable son las definiciones DDL específicas del gestor. Sigue la implementación, donde se crea la base de datos en el gestor elegido y se pobla con datos iniciales o migrados; su entregable es la base de datos operativa. Finalmente, la etapa de operación y mantenimiento cubre el uso cotidiano del sistema, el monitoreo de desempeño, los respaldos y los ajustes al esquema conforme cambian los requerimientos del negocio; su entregable es la continuidad operativa del sistema a lo largo del tiempo.

6. Sistema gestor de bases de datos (SGBD)

Un sistema gestor de bases de datos es el software que media entre los usuarios y aplicaciones, por un lado, y la base de datos física, por otro, y que provee los mecanismos para definir, construir, manipular y compartir los datos de forma controlada (Silberschatz et al., 2020). Ramakrishnan y Gehrke (2003) describen su arquitectura interna como un conjunto de componentes organizados en capas: el procesador de consultas, que interpreta y optimiza las instrucciones recibidas y decide el plan de ejecución más eficiente; el gestor de almacenamiento, que traduce operaciones lógicas en accesos al disco a través de un administrador de buffers e índices; el gestor de transacciones, que garantiza atomicidad, consistencia, aislamiento y durabilidad; y el gestor de respaldo y recuperación, que restaura el sistema a un estado consistente tras una falla.

Esta arquitectura modular no es estática: la investigación reciente explora sustituir o asistir a sus componentes con aprendizaje automático. Kraska et al. (2018) proponen tratar el índice, pieza del gestor de almacenamiento, como un modelo que predice la posición de un registro dentro de un arreglo ordenado, y reportan que sus índices aprendidos superan hasta en 70% en velocidad a los árboles B optimizados en caché y reducen en un orden de magnitud el consumo de memoria.

Sobre el procesador de consultas, Marcus et al. (2019) presentan Neo, un optimizador basado en redes neuronales que aprende a construir planes de ejecución a partir de la retroalimentación de ejecuciones reales, y Marcus et al. (2021) proponen Bao, que conserva el optimizador tradicional del gestor y solo aprende a elegir, para cada consulta, entre un conjunto reducido de sugerencias de ejecución.

Un SGBD expone distintos sublenguajes según la operación que se requiera realizar. El DDL (Data Definition Language, lenguaje de definición de datos) se usa para crear y modificar la estructura de la base de datos —sentencias como CREATE TABLE o ALTER TABLE—. El DML (Data Manipulation Language, lenguaje de manipulación de datos) se usa para consultar e insertar, actualizar o eliminar datos —SELECT, INSERT, UPDATE, DELETE—. El DCL (Data Control Language, lenguaje de control de datos) administra permisos y privilegios de acceso —GRANT y REVOKE—. A estos se suma el control de transacciones, con instrucciones como COMMIT y ROLLBACK, que delimitan el alcance de una operación atómica. Los SGBD se clasifican, entre otros criterios, según su modelo de datos (relacional, orientado a documentos, de grafos, clave-valor) y según el número de usuarios y sitios que soportan (centralizados frente a distribuidos).

7. Sistema de base de datos: arquitectura de tres niveles

El estándar propuesto por el comité ANSI-SPARC, retomado por Date (2004) y por Elmasri y Navathe (2016), organiza un sistema de base de datos en tres niveles de abstracción. El nivel interno (o físico) describe cómo se almacenan realmente los datos: estructuras de archivos, índices, rutas de acceso. El nivel conceptual describe qué datos existen y cómo se relacionan entre sí, sin entrar en detalles de almacenamiento físico ni de cómo un usuario particular los visualiza; es la representación global de toda la base de datos. El nivel externo (o de vistas) describe cómo cada grupo de usuarios o aplicación particular percibe únicamente la porción de datos que le concierne, mediante vistas personalizadas.

Esta arquitectura de tres niveles es lo que hace posible la independencia de datos: la independencia física permite modificar el nivel interno (por ejemplo, cambiar el tipo de índice usado) sin afectar el nivel conceptual ni las aplicaciones; la independencia lógica permite modificar el nivel conceptual (por ejemplo, agregar una nueva tabla) sin afectar las vistas externas que no dependen de ese cambio. Los índices aprendidos de Kraska et al. (2018) ilustran bien la independencia física: al pertenecer al nivel interno, pueden sustituir a un árbol B sin que el usuario perciba cambios en las tablas ni en las consultas que ya utilizaba.

En cuanto a la arquitectura de despliegue, un sistema centralizado concentra el SGBD y los datos en un único servidor, mientras que una arquitectura cliente-servidor separa el procesamiento en un servidor que aloja el SGBD y los datos, y clientes que envían solicitudes a través de la red y reciben los resultados, lo que permite escalar el número de usuarios sin duplicar el gestor en cada máquina. Este es el esquema bajo el cual Marcus et al. (2021) evalúan Bao, con motores comerciales alojados en entornos de nube.

8. Modelos de datos

Un modelo de datos es un conjunto de conceptos y notaciones que permite describir la estructura de una base de datos: qué datos contiene, cómo se relacionan y qué restricciones deben cumplir (Elmasri y Navathe, 2016). Su importancia radica en que actúa como el lenguaje común entre el diseñador, el desarrollador y el negocio para representar la realidad de forma que pueda implementarse en un sistema gestor. Los modelos de datos se clasifican, en primer lugar, por su nivel de abstracción: los modelos conceptuales de alto nivel, como el modelo entidad-relación, describen la realidad en términos cercanos a como la perciben los usuarios, usando entidades y relaciones sin detalles de implementación; los modelos lógicos o de implementación, como el modelo relacional, el modelo orientado a objetos o el modelo de documentos, describen los datos en los términos que un SGBD concreto puede procesar; y los modelos físicos describen cómo se almacenan los datos a nivel de archivos, bloques e índices.

En segundo lugar, los modelos se clasifican por su estructura o forma de organizar los datos. El modelo relacional organiza los datos en tablas (relaciones) compuestas por filas y columnas, vinculadas mediante llaves; es el más usado en sistemas transaccionales por su solidez teórica basada en el álgebra relacional. El modelo jerárquico organiza los datos como un árbol, donde cada registro tiene un único padre. El modelo de red generaliza al jerárquico permitiendo que un registro tenga múltiples padres, mediante un grafo. Los modelos NoSQL más recientes incluyen el modelo de documentos (datos semiestructurados agrupados en documentos, como en MongoDB), el modelo clave-valor (pares simples optimizados para lectura y escritura rápida, como en Redis) y el modelo de grafos (nodos y aristas, óptimo para representar relaciones densas, como en Neo4j). La elección del modelo depende del tipo de datos, del patrón de acceso esperado y de los requisitos de consistencia del sistema que se está diseñando.

Referencias

Coronel, C., & Morris, S. (2019). *Database systems: Design, implementation, and management* (13a ed.). Cengage Learning.

Date, C. J. (2004). *An introduction to database systems* (8a ed.). Pearson Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7a ed.). Pearson.

Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. En *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 489–504). Association for Computing Machinery. https://doi.org/10.1145/3183713.3196909

Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M., & Kraska, T. (2021). Bao: Making learned query optimization practical. En *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)* (pp. 1275–1288). Association for Computing Machinery. https://doi.org/10.1145/3448016.3452838

Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O., & Tatbul, N. (2019). Neo: A learned query optimizer. *Proceedings of the VLDB Endowment, 12*(11), 1705–1718. https://doi.org/10.14778/3342263.3342644

Ramakrishnan, R., & Gehrke, J. (2003). *Database management systems* (3a ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7a ed.). McGraw-Hill.